LightWork AI - Data Engineer Take-Home Submission

This document explains how I would build a simple, reliable, and scalable data platform for LightWork AI. The aim is to support daily operations, live communication flows, and AI-driven actions.

What this submission covers

- How to bring in PMS data from old and new systems
- How to process messages, calls, and transcripts in real time
- How AI can use this data safely and usefully
- How the platform can grow without becoming too expensive

Review focus

- Reliability: no silent data loss and safe retries
- Security: protect tenant data and keep access controlled
- Practicality: use tools a startup can run with a small team
- Performance: watch speed, quality, and cost together

> **Repository Link:** https://github.com/egenc/lightwork-data-engineer-takehome

1) Problem and Design Goals

LightWork needs one platform that can collect data from many PMS systems, handle live events, and give AI tools the right context to take useful action.

> **Why this matters:** If the platform is slow, messy, or hard to trust, the AI layer will also be slow, messy, and hard to trust.

Primary goals

- Bring in PMS data from both legacy and modern systems without losing records
- Support both scheduled data syncs and live event processing
- Protect tenant data and support compliance needs
- Scale step by step with clear limits for cost and speed

Guiding principles

- Start with proven tools and only add complexity when it solves a real problem
- Make replay, safe retries, and monitoring part of the design from the start
- Keep the system easy to understand for both engineers and operations teams

2) Architecture for PMS Ingestion and Storage

> **Why this matters:** PMS data is the business backbone. If this layer is unreliable, everything above it becomes unreliable too.

Ingestion approach

- **Connector workers (Python):** one small service for each PMS provider
- **Legacy APIs:** scheduled polling, checkpoints, retries, and backoff
- **Modern APIs:** webhooks, signature checks, and regular reconciliation
- **Transport:** Kafka topics so events are durable and can be replayed

Medallion data model

Bronze (Raw)

Raw source payloads are stored exactly as they arrive. This makes auditing and replay possible.

Silver (Conformed)

Data is cleaned, deduplicated, and checked against policy rules. Core tenancy data lives in PostgreSQL and searchable text goes to OpenSearch.

Gold (Serving)

Business-ready views combine structured and unstructured data for AI actions, dashboards, and reporting.

Data stores and purpose

- **PostgreSQL:** main source of truth for tenants, tenancies, and actions
- **OpenSearch:** fast search across messages and transcripts
- **Object storage:** low-cost storage for raw files, audio, and payloads
- **Redis (optional):** fast cache and short-lived coordination state

3) Real-Time Processing and AI Orchestration

> **Why this matters:** Some issues cannot wait for a nightly sync. Urgent messages, calls, and transcript signals may need action within seconds.

Live data flow

- Messages, calls, and webhooks enter a real-time ingest API
- Events are sent to Kafka real-time topics
- Processors add tenant and conversation context
- AI workers classify intent, urgency, and next best action
- An action engine sends recommendations or triggers approved workflows

Airflow + Kubernetes responsibilities

Airflow

- Schedules PMS sync and reconciliation jobs
- Runs backfills in a controlled way
- Gives the team visibility into scheduled workflows

Kubernetes

- Runs connector, consumer, and AI worker services
- Scales workloads up or down based on lag and latency
- Keeps heavy workloads isolated so one problem does not slow down everything else

Performance targets

- 95% of ingest-to-classification events should finish in under 3 seconds
- 95% of ingest-to-recommendation events should finish in under 8 seconds
- The full path should be measurable from ingest to AI action

4) Compliance, Benchmarking, and Scale Plan

> **Why this matters:** A system is not production-ready if it is fast but unsafe, or smart but too expensive to run.

Compliance and governance (current offering)

- Encryption in transit and at rest
- Tenant-level access controls and audit trails
- PII-aware prompt handling with masking rules
- Retention rules by data type such as events, transcripts, and raw media

AI quality and safety benchmarking

- **Offline:** measure accuracy, ranking quality, and safety issues
- **Online:** use shadow mode and controlled tests before full rollout
- **Operations:** track speed, model cost per case, and drift over time

Scale interventions (when to act)

- Add more consumers when queue lag stays too high for too long
- Add Redis when repeated hot reads start slowing user-facing requests
- Split high-volume tables and move older data to cheaper storage
- Adopt dedicated vector infrastructure when semantic search becomes too slow

> **Implementation posture:** Start simple, keep it safe, measure everything important, and scale only when the data shows it is necessary.